Informe Técnico - Fase 2

FlowDeploy - Pipeline CI/CD automatizado para despliegue continuo

Proyecto:	FlowDeploy – Pipeline CI/CD automatizado con despliegue continuo
Fase:	Fase 2 – Repositorio, Integración continua y calidad
Equipo:	SN-1
Versión:	1.0
Fecha de elaboración:	11 de noviembre de 2026
Repositorio:	https://github.com/herberth-barrera/FlowDeploy-ESIT
Tutor:	Ing. Carlos G. Rodriguez
Ciclo:	02 - 2026
Integrante responsable de este informe:	Juan Francisco Azucena Mejía - Ingeniero DevOps Jr.

Equipo: Herberth Barrera (líder / tech lead) · Andrew Chicas (desarrollo de aplicación) · Juan F. Azucena (DevOps) · Diego Alfaro (infraestructura) · Héctor Benjamín Chipagua Calles (QA / documentación).

Integrantes y roles

Integrante	Rol
Herberth Barrera	**Líder**
Juan Francisco Azucena Mejía	**DevOps Jr.**
Diego Alfaro	**Infraestructura Jr.**
Andrew Chicas	**Desarrollador Jr.**
Héctor Benjamín Chipagua Calles	**QA/Analista**

1. Resumen ejecutivo

Durante la Fase 2 se implementó y validó el pipeline de integración continua del proyecto sobre la aplicación Nova Campus, sustituyendo el workflow esqueleto de la Fase 0 por un pipeline funcional que ejecuta análisis estático, pruebas automatizadas y verificación de arranque sobre cada cambio propuesto.

El pipeline fue validado en tres escenarios: sobre sí mismo (PR #4, ejecución 42548599241), sobre el código de la aplicación (PR #2, ejecución 42549407938, 39/39 pruebas y smoke correcto) y mediante la provocación controlada de fallos (ejecuciones 42539880570 y 42542545830), que confirmaron que el pipeline detecta y reporta errores en la etapa correcta.

El resultado técnico de la fase está completo y es verificable. Persisten dos dependencias ajenas al equipo de desarrollo: la aprobación del PR #2, que impide integrar la aplicación en main, y la entrega de la bitácora oficial de la fase. Ambas se documentan en la sección 7.

2. Alcance y periodo evaluado.

En este informe se documentan exclusivamente las actividades ejecutadas entre el 19 de octubre y el 8 de noviembre de 2026 (Fase 2 del calendario oficial)

Se establecen cuatro puntos de alcance:

1. La aplicación Nova Campus es la base preexistente del proyecto. Su commit inicial (580c910) es del 17 de octubre, dentro del cierre de la Fase 1. En la Fase 2 se trabajó sobre ella (integración continua, pruebas y validación), no en su desarrollo funcional.
2. Las correcciones documentales posteriores al 8 de noviembre (este mismo informe, la versión 2 de la matriz de pruebas y la actualización de la bitácora) se identifican como tales y no se presentan como actividades del periodo evaluado.
3. El despliegue continuo CD no forma parte del alcance de esta fase. Corresponde a la Fase 3 (9 - 22 de noviembre) e incluye el deploy hook de Render, el health check posterior al despliegue y la gestión de secretos.
4. La corrección es documental: Los lineamientos del tutor indican que no es necesario repetir trabajos ya realizados correctamente. Por ello no se ejecutaron fusiones ni cambios de código posteriores al 8 de noviembre. Las únicas modificaciones de esta entrega en prórroga son documentales.

La aplicación Nova Campus fue entregada por el rol de desarrollo el 17 de octubre e incorporada al repositorio mediante el PR #2, abierto esa misma tarde. El equipo lo adoptó como base de trabajo de la Fase 2 (comunicación del 18/10, Anexo 2), de modo que la actividad de esta fase se concentró en llevar ese código a un pipeline automatizado, validarlo y documentarlo. El desarrollo de la aplicación y la implementación del pipeline corresponden a roles distintos y se encadenan.

Sin la aplicación integrada no es posible validar el pipeline contra el código real, y sin la aprobación de la fusión no es posible completar el flujo hacia main.

2.1 Base de las decisiones adoptadas

Cuando el plan recibido no cubría un aspecto, la decisión se adoptó con base en los lineamientos del proyecto y quedó documentada. Esta tabla registra esa base:

Decisión	Fundamento	Fuente
Estrategia GitHub Flow, sin rama dev	Un único flujo de despliegue continuo: equipo de cinco integrantes	Documento de Diseño (Fase 1)
Render como plataforma de despliegue	Requisito técnico RT-06: operar en plan gratuito	Documento de Análisis RT-06
Despliegue por deploy hook y no por auto-deploy nativo	El auto-deploy dejaría el pipeline de CD sin función demostrable	Documento de Diseño, reglas de disparo
Sustituir la etapa build	Runtime interpretado: package.json no define script build y el diseño no lo contempla	package.json: Documento de Diseño, sección 3
Añadir verificación de arranque (npm run smoke) al CI	Sin etapa de build, el pipeline no comprobaba que la aplicación arrancara	Decisión de implementación de Fase 2: validada por QA (CI-09): registrada como I-06
Node.js 22	engines: “>= 22 <23” en package.json	package.json
Contenerización opcional y diferida	Los lineamientos del proyecto lo marcan como opcional	Documento de proyecto (ESIT)
Validar sobre el merge commit del PR #2	Es el árbol exacto que recibiría main al fusionar	Documento de Diseño: PR#2

3. A. Aplicación web base

3.1 Identificación

Elemento	Descripción
Nombre	Nova Campus
Tipo	Aplicación web de gestión académica (estudiantes, asignaturas e inscripciones)
Runtime	Node.js 22 (engines: “>=22 <23”)

Framework	Express 5
Vistas	EJS (renderizado en servidor)
Persistencia	SQLite mediante better-sqlite3
Estructura	49 archivos versionados: 28 en src/, 8 en tests/, 3 en scripts/

3.2 Funcionalidades implementadas

Módulo	Funcionalidad	Reglas de negocio relevantes
Estudiantes	Alta, listado y cambio de estado activo/inactivo	Nombre mínimo 2 caracteres: correo con formato válido y único
Asignaturas	Alta, listado y cambio de estado activo/inactivo	Código único: créditos entre 1 y 10
Inscripciones	Registro y listado	No se permite inscripciones duplicadas; estudiantes y asignatura deben existir y estar activos: año >= 2020, ciclo académico 1 o 2
Estado del servicio	/health y /version	/health responde 200 con estado ok; /version reporta version y commit del despliegue
Interfaz	Panel, formularios y tablas con validación y mensajes de error	Los errores de validación devuelven 400 y conservan datos ingresados

Endpoints Expuestos:

Método	Ruta	Propósito
GET	/	panel principal
GET	/health	Estado del servicio (usado por el pipeline)
GET	/version	Versión y commit desplegado
GET/POST	/students	Listado y alta de estudiantes
POST	/students/:id/toggle	Cambio de estado del estudiante
GET/POST	/courses	Listado y alta de asignatura
POST	/courses/:id/toggle	Cambios de estado de la asignatura
GET/POST	/enrollments	Listado y registro de inscripciones

3.3 Estructura del código fuente

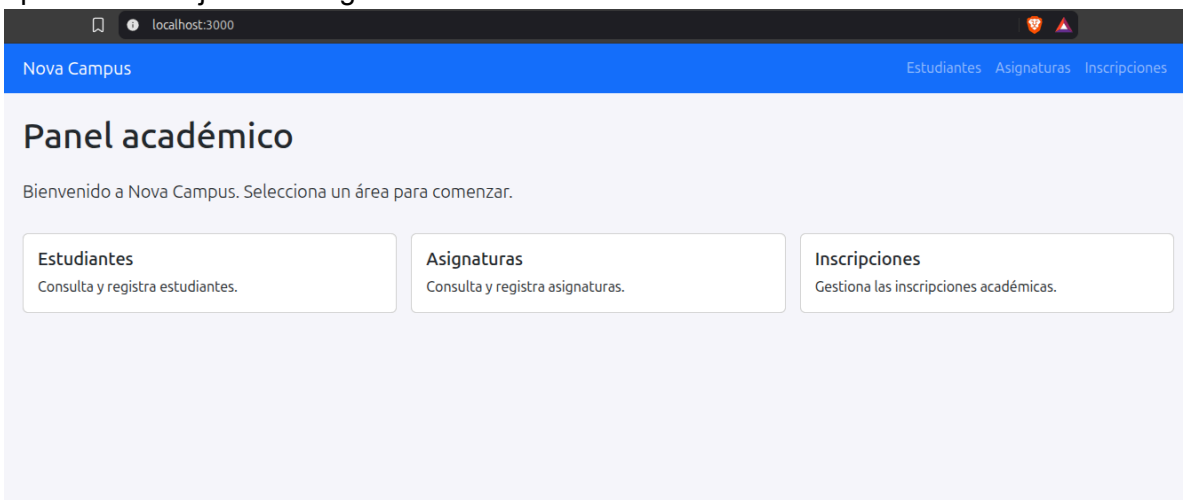
```
.
├── .github/workflows/     ci.yml, cd.yml (definición del pipeline)
```

```
├── src/
│   ├── app.js            configuración de Express y montaje de rutas
│   ├── server.js         punto de entrada (npm start)
│   ├── routes/           definición de endpoints
│   ├── controllers/      manejo de peticiones y respuestas HTTP
│   ├── services/         lógica de negocio
│   ├── validators/       reglas de validación
│   ├── db/               conexión, consultas y esquema SQL
│   ├── views/            plantillas EJS
│   └── public/           hoja de estilos
├── tests/
│   ├── integration/      5 archivos (rutas y base de datos)
│   └── unit/             3 archivos (servicios, validadores, importación de app)
├── scripts/smoke.js      verificación de arranque
├── eslint.config.js
├── package.json
└── .env.example
```

La separación modular de la aplicación facilita pruebas aisladas y eficientes: tests unitarios para la lógica de negocio (sin levantar el servidor) y tests de integración para el ciclo completo de los endpoints.

3.4 Evidencia de ejecución

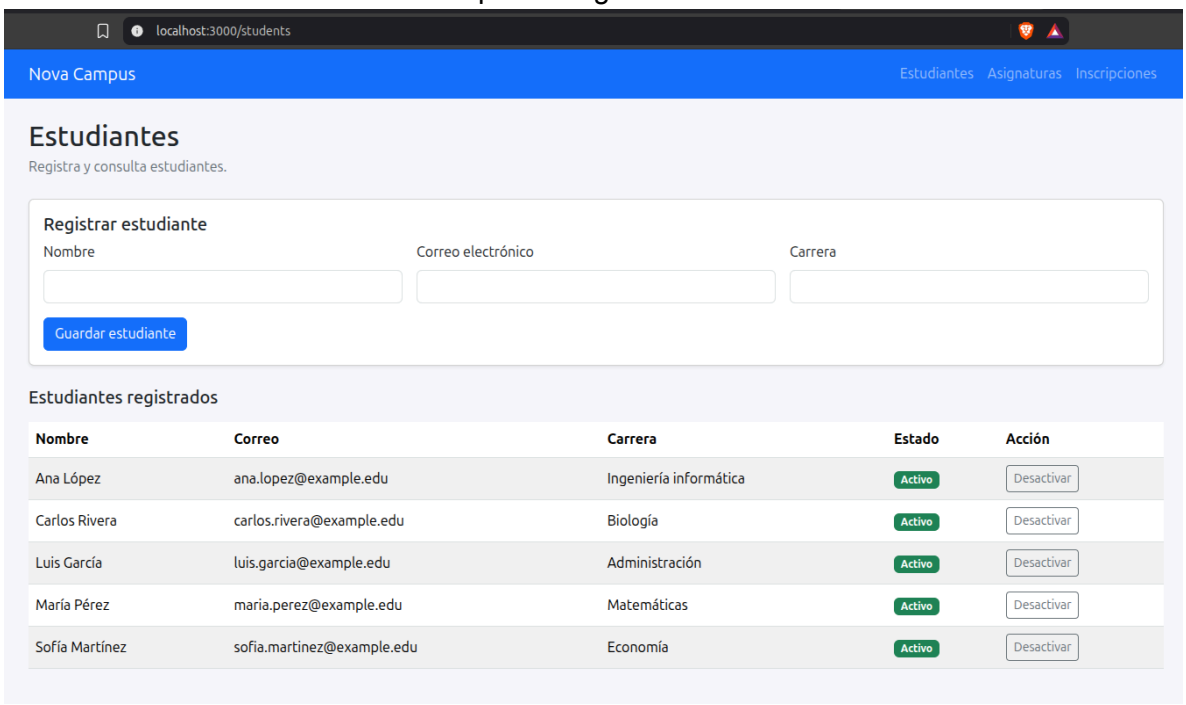
Aplicación en ejecución Fig. 1:



Get /health devolviendo 200 con estado ok Fig. 2:

```
devfrankmj@devfrankmj-System-Product-Name:~/FlowDeploy-ESIT$ curl -i localhost:3
000/health
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 15
ETag: W/"f-VaSQ4oDUiZblZNAEkkN+sX+q3Sg"
Date: Tue, 10 Nov 2026 05:13:25 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Listado de estudiantes con datos de pruebaFig. 3:



Verificación	Comando/ruta	Resultado obtenido
Análisis estático	npm run lint	Sin errores ni advertencias (código de salida 0)
Pruebas automatizadas	npm test	39 pruebas, 39 correctas, 0 fallidas
Verificación de arranque	npm run smoke	/health responde 200 con status: “ok”, / responde 200 e incluye el panel académico (el script falla si alguno difiere)

El script de smoke no imprime el código de estado cuando la verificación es correcta, solo informa el resultado. Por eso la evidencia del código 200 se aporta además con una consulta directa (Get /health devolviendo 200 con estado ok imagen) y no únicamente con

la salida del script.

Reproducir:
curl -i localhost:3000/health
npm run smoke

3.5 Instalación y ejecución

```
# 1. Obtener el código
git clone https://github.com/herberth-barrera/FlowDeploy-ESIT.git
cd FlowDeploy-ESIT
git checkout feature/nova-campus

# 2. Verificar el runtime (prerrequisito)
node -v                 # debe reportar v22.x

# 3. Instalar dependencias de forma reproducible
npm ci

# 4. Preparar el directorio temporal de las pruebas
mkdir -p work           # obligatorio solo para npm test (ver nota)

# 5. Opcionales: configuración explícita y datos de demostración
cp .env.example .env    # opcional: la app trae valores por defecto
npm run seed            # opcional: carga datos de ejemplo

# 6. Ejecutar
npm start               # http://localhost:3000
```

Cinco de los pasos son obligatorios:
obtener el código, verificar el runtime, instalar dependencias, crear work/ antes de las pruebas e iniciar la aplicación. Los dos marcados como opcionales no alteran el funcionamiento
.env solo hace explícitos valores que el código ya define por defecto (port=3000, DB_PATH=./data/nova-campus.db, APP_VERSION=dev, GIT_SHA=unknown), y seed únicamente carga datos de demostración.

Verificación posterior (implementa la misma secuencia que el Pipeline):
npm run lint
npm test
npm run smoke

Nota sobre work/: los tests crean directorios temporales dentro de work/, que está excluido en .gitignore (en la línea 227). En un clon limpio esa carpeta no existe y la suite falla con ENOENT. El comando mkdir -p work es un workaround temporal, documentado como defecto D-01: el pipeline lo ejecuta como paso previo a las pruebas. No es necesario para npm start: la aplicación arranca sin esa carpeta.

Scripts disponibles en package.json: lint, test, smoke, seed, dev, start. No existe script build. (se explica en la sección 4.3).

4. B Pipeline de integración continua

4.1 Archivo YAML

Ubicación: .github/workflows/ci.yml. Enlace directo a la versión validada: https://github.com/herberth-barrera/FlowDeploy-ESIT/blob/9b4e66b/.github/workflows/ci.yml

```yaml
name: CI - Integracion Continua

on:
  push:
    branches: [ main ]
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  validacion:
    name: Validacion del codigo
    runs-on: ubuntu-latest
    timeout-minutes: 10

    steps:
      - name: Checkout del repositorio
        uses: actions/checkout@v4

      - name: Configurar Node.js 22
        uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: 'npm'

      - name: Instalar dependencias
        run: npm ci

      # Workaround temporal (incidencia D-01): la suite crea directorios
      # temporales en work/, excluido por .gitignore, y falla en clon limpio.
      - name: Preparar directorio de trabajo
        run: mkdir -p work

      - name: Lint
        run: npm run lint

      - name: Pruebas
        run: npm test

      - name: Smoke test de la app
        run: npm run smoke
```

4.2 Etapas de pipeline

Etapa	Comando	Propósito	Criterio de fallo
Preparación	actions/checkout + setup-node	Obtener el código y fijar Node 22 con caché de npm	Error de descarga o versión distinta de 22.
Instalación	npm ci	Instalación reproducible desde package-lock.json	Dependencias no instaladas o lockfile inconsistente
Preparación del entorno	mkdir -p work	Crea el directorio temporal requerido por la suite (D-01)	—
Validación	npm run lint	Análisis estático con ESLint	cualquier error de lint (código ≠ 0)
Pruebas	npm test	39 pruebas unitarias y de integración (node –test)	Cualquier prueba fallida (código de error ≠ 0)
Verificación de arranque	npm run smoke	Levanta la app y comprobar /health y /	Respuesta distinta de 200

4.3 Etapas del CI respecto al Documento de Diseño

El Documento de Diseño define cinco pasos para el CI: checkout, configuración del entorno, instalación de dependencias, lint y pruebas. Excluye de forma explícita la etapa build: “No he colocado etapa de build. El Dev no define script de compilación, por lo que el CI valida solo con lint y pruebas.” La verificación mediante /health se ubica en el diseño dentro del CD, como paso posterior al despliegue.

La implementación respeta esa estructura y añade dos pasos:

1. mkdir -p work - crea el directorio temporal que la suite requiere en un clon limpio (workaround del defecto D-01)
2. npm run smoke - verificación de arranque que no estaba prevista en el diseño: levanta la aplicación en el runner y comprueba que /health y / respondan 200.

La justificación del segundo: al no existir etapa de build, el pipeline verifica estilo y pruebas, pero no que la aplicación efectivamente arranca. El smoke cubre ese hueco sin introducir una compilación inexistente y adelanta, de forma controlada, la verificación que el Diseño reservó para el CD. El ajuste se documenta como desviación de implementación (I-06) y fue validado de forma independiente por QA (caso CI-09)

4.4 Disparadores configurados

Evento	Rama	Efecto
push	main	Valida el estado integrado de la rama principal tras cada fusión.
pull_request	Cualquier rama	Valida la fusión propuesta antes de que se apruebe

Los cambios en ramas feature/* o ci/* no disparan una ejecución por push: la validación ocurre en el pull_request asociado, sobre el merge commit que es exactamente el árbol que recibiría main si se aprueba la fusión.

4.5 Ejecuciones del workflow

Run	Fecha	Evento	Rama	Resultado	Duración
42334346557	06/11 21:30	push	ci/validacion	Correcta - primera validacion real (lint + 39/39)	12 s
42539766290	08/11 18:21	push	qa/ci-06-lint	Correcta (línea base)	18 s
42539880570	08/11 18:22	push	qa/ci-06-lint	fallida - Lint	11 s
42540465627	08/11 18:29	push	qa/ci-06-lint	Correcta (infraccion retirada)	14 s
42542545830	08/11 18:52	push	qa/ci-06-lint	Fallida - pruebas	16 s
42543095483	08/11 18:59	push	qa/ci-06-lint	Correcta (prueba retirada)	15 s
42543910619	08/11 19:08	push	qa/ci-06-lint	Correcta (incluye npm run smoke)	11 s
42548599241	08/11 20:01	pull_request	ci/pipeline-real-node22-final	Correcta — el workflow validándose	13 s
42549407938	08/11 20:10	pull_request	feature/nova-campus (PR #2)	Correcta — 39/39 + smoke	15 s

El repositorio acumula 13 ejecuciones del workflow de CI. Nueve ejecutaron validación real y se listan arriba; las otras cuatro corresponden al esqueleto de la Fase 0 y se detallan en la siguiente nota:

Nota sobre las ejecuciones del esqueleto: Hasta el 08/11, las ejecuciones que corrían sobre main, sobre el PR #2 y sobre el PR #3 correspondian al workflow esqueleto de la fase 0, que no ejecuta lint, pruebas ni verificación de arranque: 36813327863 (30/09, main), 39095047724 (17/10, PR #2), 42527278490 (08/11 15:51, PR #3) y 42532019645 (08/11 16:48, PR #2).

La primera validación real del pipeline ocurrió el 06/11 sobre la rama temporal ci/validacion (42334346557), sin afectar a main ni al PR #2: el pipeline funcional no llegó a la rama de la aplicación hasta el 08/11 a las 20:10, con la fusión del PR #4. Por eso el primer check con validación sobre el PR #2 es el 42549407938.

Las ejecuciones del esqueleto se documentan como estado previo del repositorio y no se cuentan como evidencia de validación; esa condición se describe en el defecto D-03.

2 ejecuciones a destacar:

- 42548599241 - el PR que introduce el pipeline se valida con ese mismo pipeline sobre el merge commit. Es la comprobación de que el workflow es correcto y está bien integrado.
- 42549407938 — la validación real de la aplicación dentro del PR #2: lint sin errores, 39/39 pruebas y smoke correcto.

4.6 Ejecuciones con error y su desarrollo

Los dos fallos del registro fueron provocados deliberadamente para verificar que el pipeline detecta errores. No son defectos del código entregado.

Run	Error introducido	Etapa que fallo	Resultado observado	Corrección aplicada
42539880570	Sentencia debugger en src/qa-ci-06.js	Lint	ESLint reportó la regla infringida; el job terminó con código 1	Se retiró el archivo; la ejecución 42540465627 volvió a verde
42542545830	aserción inválida en tests/unit/ci-08.test.mjs	Pruebas	40 pruebas ejecutadas: 39 correctas, 1 fallida; código de salida 1	Se eliminó la prueba temporal; la ejecución 42543095483 volvió a verde

Observación de alcance: estas ejecuciones demuestran que el pipeline detecta fallos. La verificación de que un fallo bloquee la fusión (protección de rama con check requerido) corresponde a la configuración de main y se documenta como pendiente en la sección 9 (RT-08).

5. C. Pruebas automatizadas

5.1 Suite existente

La aplicación cuenta con 39 pruebas distribuidas en 8 archivos:

Archivo	Tipo	Cobertura
tests/integration/students.test.js	Integración	Alta, listado, validaciones y cambio de estado de estudiantes
tests/integration/courses.test.js	Integración	Alta, listado, duplicados, rango de créditos y cambio de estado
tests/integration/ enrollments.test.js	Integración	Registro, duplicados, claves foráneas y estados inactivos

tests/integration/health-version.test.js	Integración	/health y /version
tests/integration/database.test.js	Integración	Restricciones de integridad de la base
tests/unit/services.test.js	Unitaria	Reglas de negocio de las tres entidades
tests/unit/validators.test.js	Unitaria	Límites de validación (longitud, créditos, año, ciclo)
tests/unit/app-import.test.js	Unitaria	La importación de la app no crea la base de datos

Resultado de la última ejecución: 39 pruebas, 39 correctas, 0 fallidas, 0 omitidas.

5.2 Matriz de casos de pipelines

La matriz completa, con precondiciones, resultados esperados y obtenidos, evidencias y numeración por caso, se adjunta como Anexo 1 (elaborada por QA). La tabla siguiente es un resumen de esa matriz y no la sustituye.

Caso	Objetivo	Resultado	Evidencia
CI-01	Ejecución al abrir o actualizar un PR	Verificado con el pipeline real	42549407938
CI-02	Ejecución por push a main	Pendiente: requiere la fusión del PR #2	
CI-03	Comportamiento en push a ramas feature/*	Verificado: la validación ocurre en el PR asociado	42549407938
CI-04	Instalación reproducible con Node 22 y npm ci	Verificado	42549407938
CI-05	Lint sin errores	Verificado	42549407938
CI-06	Lint detecta una infracción	Verificado	42539880570 (rojo) / 42540465627 (verde)
CI-07	Pruebas correctas (39/39)	Verificado	42549407938
CI-08	Una prueba fallida detiene el pipeline	Verificado	42542545830 (rojo) / 42543095483 (verde)
CI-09	Smoke test de arranque	Verificado	42543910619
CI-10	Etapa de build	No aplica - el Diseño excluye “build” (4.3)	
CI-11	Clon limpio sin preparación manual	Fallido — 23 de 39 fallan, 16 pasan	Ejecución local

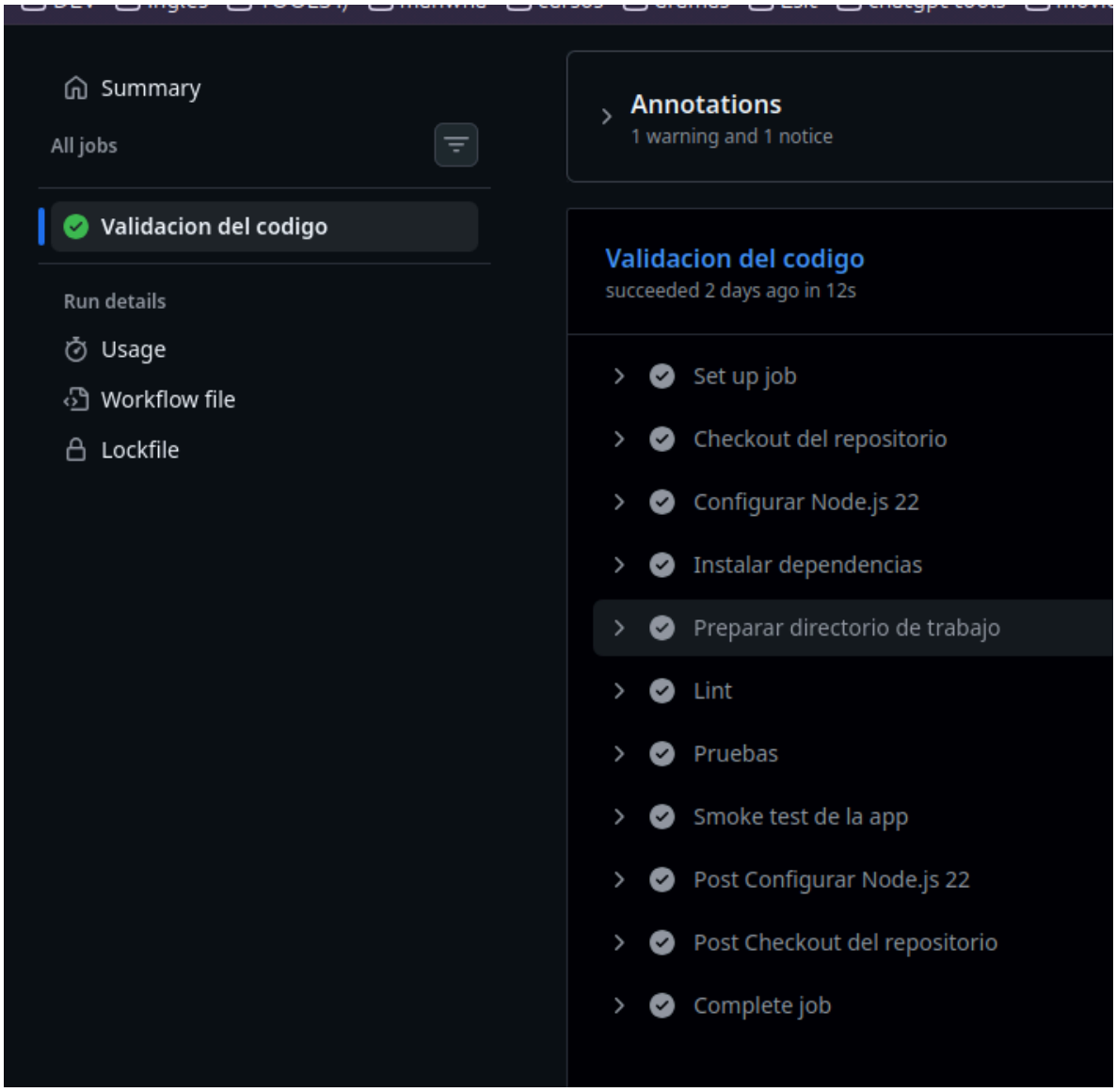
5.3 Defectos e incidencias detectadas

ID	Tipo	Descripción	Severidad	Responsable	Estado
D-01	Defecto	Los tests fallan en clon limpio (ENOENT mkdtemp): work/ está en	Alta	Desarrollo de aplicación	Abierto — workaround en ci.yml; corrección

		.gitignore:227 y la suite no crea el directorio			prevista en Fase 3
D-02	Bloqueo	La aplicación no está en main: PR #2 sin revisión tras tres solicitudes (22/10, 05/11, 07/11); la fusión no se ejecutó	Crítica	Líder de equipo (aprobación)	Abierto - impacto en Fase 3 (CI en main, RT-08)
D-03	Defecto	El ci.yml de main es el esqueleto de la Fase 0 (solo “echo”)	Alta	DevOps	Mitigado - pipeline real en la rama de la aplicación; pendiente en main
D-04	No es defecto	/version devuelve dev /unknown sin variables de entorno			Cerrado - comportamiento esperado; se configura APP_VERSION y GIT_SHA en el despliegue

5.4 Evidencias de ejecución

Resumen y pasos del run 42549407938 (ejecución #14) en verde: evento pull_request, rama feature/nova-campus, 15 s, con el job desplegado y los siete pasos visibles. Fig. 4:



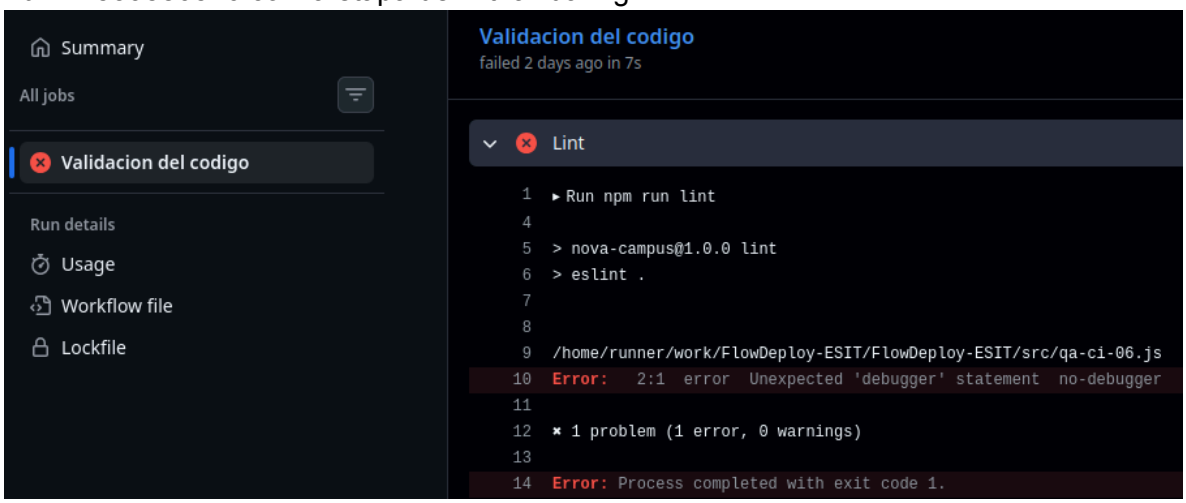
Log de pruebas del run 42549407938: # tests 39 · # pass 39 · # fail 0 Fig. 5



Log del smoke test en el mismo run Fig. 6



Run 42539880570 con la etapa de lint fallida Fig. 7:



Run 42542545830 con la etapa de pruebas fallida Fig. 8:



6. D. Repositorio de Github

6.1 Enlace y estrategia de ramas

Repositorio: https://github.com/herberth-barrera/FlowDeploy-ESIT

Rama	Propósito	Estado
main	Rama principal protegida; única rama desplegable	Activa
feature/nova-campus	Desarrollo de la aplicación	Activa (PR #2)
ci/pipeline-real-node22-final	Implementación del pipeline real (PR #4)	Fusionada y eliminada
qa/ci-06-lint	Pruebas negativas de QA	Temporal

Estrategia: GitHub Flow — main protegida (ver nota en 7.1) y ramas cortas por propósito (feature/*, ci/*,docs/*, qa/*). No existe rama dev.

6.2 Commits relevantes de la fase

Se incluyen los commits que sostienen las evidencias de la fase, incluidos los de fases anteriores que constituyen su base. La columna «Periodo» indica a qué fase corresponde cada uno; ninguno se reasigna.

Commit	Fecha	Descripción	Periodo

8d97e62	30/09 21:50	Esqueleto de ci.yml y cd.yml	Fase 0
2eb85ab	30/09 22:03	Fusión del PR #1 (Fase 0 DevOps)	Fase 0
580c910	17/10 21:40	Aplicación Nova Campus (base sobre la que opera el pipeline)	Fase 1
ea4b26d	06/11 21:30	Primer pipeline funcional, validado sobre la rama temporal ci/validacion	Fase 2
e1b9bf5	08/11 19:48	Pipeline definitivo, cabeza del PR #4	Fase 2
9b4e66b	08/11 20:10	Fusión del pipeline en la rama de la aplicación (squash del PR #4)	Fase 2
271b7ea	08/11	Incorporación del paso de smoke en la rama de pruebas de QA	Fase 2

6.3 Pull Requests

PR	Título	Base ← Head	Estado	Revisión
#1	Fase 0: workflows CI/CD	main ← feature/fase0-devops	Fusionado (30/09 22:03)	Aprobado
#2	Aplicación Nova Campus	main ← feature/nova-campus	Abierto	0 revisiones, 0 comentarios (más de 21 días)
#4	Pipeline real Node 22	feature/nova-campus ← ci/pipeline-real-node22-final	Fusionado (08/11, squash)	Check verde + fusión
#3	Revisión inicial del documentacion	main ← docs/qa-readme	Abierto	Sin revisión - pendiente de cierre/aprobación

6.4 Evidencia de integración de componentes

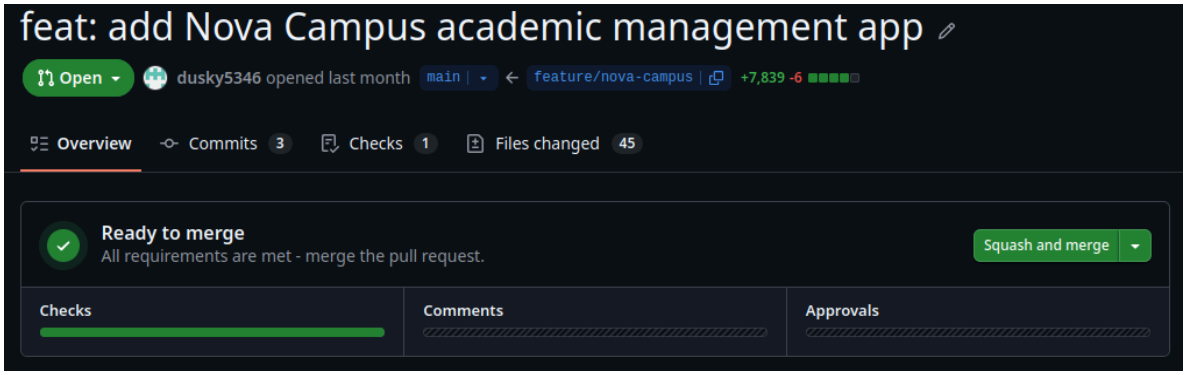
La integración de los componentes desarrollados (aplicación + pipeline) se verifica en la ejecución 42549407938, que se ejecuta sobre el merge commit del PR #2: contiene la aplicación completa y el pipeline definitivo, y ambos funcionan juntos (lint, 39/39 pruebas y smoke correcto).

Al tratarse de un evento pull_request, GitHub ejecuta la validación sobre un merge commit temporal que combina la rama base (main) con la rama del PR. Su árbol es el mismo que quedaría en main al fusionar, siempre que no haya conflictos y que main no reciba cambios nuevos entre la ejecución y la fusión. Ambas condiciones se cumplen en este caso: el PR figura como fusionable y, al momento de la ejecución, main no registraba modificaciones desde el 30/09 (2eb85ab). Por eso el resultado del run es evidencia del estado integrado, y no solo de la rama de trabajo.

Nota sobre el PR #4. El flujo definido en el Documento de Diseño establece que los PR se dirigen a main y requieren al menos una aprobación. Este PR se dirigió a la rama de la

aplicación y se fusionó sin revisión, como excepción deliberada: main no admitía aún la fusión del proyecto y el pipeline no podía publicarse en ninguna rama sin esa integración. Se registra como desviación I-07. Se respetaron las demás reglas del diseño: el cambio entró por Pull Request, el CI estaba en verde al momento de la fusión y la rama de trabajo se eliminó después.

Check del PR #2 con validación del código en verde Fig. 9:



7. Bloqueos y dependencias

7.1 Revisión del PR #2

Fecha	Hecho	Evidencia
17/10 21:40	Commit base de la aplicación	580c910
17/10 21:50	Apertura del PR #2 (main ← feature/nova-campus), 45 archivos, +7839/−6	https://github.com/herberth-barrera/FlowDeploy-ESIT/pull/2
22/10 20:46	Primera solicitud de fusión, planteada para poder abrir el PR del pipeline	C-04
05/11 19:13	Segunda solicitud de revisión, dirigida al equipo de desarrollo	C-06
07/11 14:11	Tercera solicitud: sin la fusión no es posible publicar el pipeline. QA consulta al líder de forma directa	C-07
08/11 16:15	QA pregunta si solo el líder puede aprobar los PR; en el grupo se responde afirmativamente	C-08
08/11 16:48	Único check asociado al PR hasta ese momento: workflow esqueleto de la Fase 0 (echo, 6 s)	Run 42532019645
08/11 19:48	Push del pipeline definitivo en la rama ci/pipeline-real-node22-final	Commit e1b9bf5
08/11 20:01	El PR #4 se valida con el propio pipeline	Run 42548599241
08/11 20:10	Fusión del PR #4 en la rama de la aplicación; el PR #2 revalida con el pipeline real	9b4e66b; run 42549407938

09/11 19:39	El rol de DevOps expone en el grupo que la fusión no es viable en esta corrección: el trabajo se construyó sobre el PR de desarrollo y el alcance solicitado por el tutor es documental	C-09
17/10 - 08/11	0 revisiones, 0 comentarios, 0 revisores solicitados (más de 21 días), pese a tres solicitudes registradas	Historial de revisiones del PR #2; Anexo 2

Nota sobre la protección de main. El 08/11 se afirmó en el grupo que solo el líder podía aprobar los PR. Al cierre de este informe, la página del PR #2 muestra el estado «Ready to merge» con la sección de aprobaciones vacía, por lo que el alcance exacto de esa restricción no pudo verificarse desde el repositorio. Se deja registrado como punto a confirmar en la Fase 3, junto con RT-08.

Impacto: el flujo completo (fusión a main → despliegue) no se ejecutó dentro del periodo. La validación se realizó sobre el merge commit del PR #2, técnicamente equivalente al árbol que recibiría main.

Estado de la dependencia: el PR #2 continúa sin revisión y la fusión no se ejecutó. En consecuencia, la evidencia de CI instalado en main y la activación de RT-08 se trasladan a la Fase 3. Este informe no declara cumplidos esos dos puntos.

RT-08 (Documento de Análisis, sección 5): "La rama principal deberá tener protección que bloquee la fusión si el CI falla." Su verificación requiere que el pipeline esté instalado en main y declarado como check requerido; ambas condiciones dependen de la fusión del PR #2.

Registro posterior al periodo. El 11/11: main recibió dos commits directos (30e8fb0, 42c2d7e), posteriores al periodo evaluado y ajenos a este informe. Las ejecuciones de CI que generaron corresponden al workflow esqueleto de la Fase 0 y no ejecutan validación (ver D-03). El PR #2 continúa sin revisión.

7.2 Inconsistencias detectadas en la comunicación del plan

La asignación de tareas recibida el 2 de noviembre (C-05, Anexo 2) corresponde a la Fase 3 y describe un producto distinto del implementado. Las comunicaciones se citan con los códigos definidos en el Anexo 2; la cita entre comillas reproduce el texto original.

ID	Inconsistencia	Evidencia
I-01	Identifica la aplicación como Calculadora de Cotizaciones con endpoint /api/quote; el producto implementado es Nova Campus (/students, /courses, /enrollments)	C-05: "la Calculadora de Cotizaciones", "los endpoints (/health y /api/quote)"
I-02	Referencia ci-cd.yml con etapas Lint, Build y Deploy; el repositorio usa ci.yml con lint, pruebas y verificación de arranque	C-05: "(ci-cd.yml)", "las etapas de Lint, Build y Deploy"
I-03	Solicita "empezar a subir el código a GitHub"	C-05: "empiecen a subir el

	el 02/11; el PR #2 estaba abierto desde el 17/10 y main contenía la Fase 0 desde el 30/09	codigo que tengan a github"; PR #2; 2eb85ab
I-04	La asignación se rotula Fase 3 pero se envía durante el periodo de Fase 2	C-05: "unas tareas para la fase 3", enviada el 02/11
I-05	El plan del 06/10 asigna el diseño conceptual del pipeline y el análisis de la aplicación, sin asignar explícitamente el desarrollo de la aplicación ni la implementación funcional del pipeline; ambas se ejecutaron y se adoptaron como base de la fase	C-01 (asignación del 06/10); C-03 (adopción como base de Fase 2); PR #2; PR #4
I-06	La implementación del CI añade los pasos mkdir -p work y npm run smoke, no previstos en el Documento de Diseño, que define cinco etapas y excluye build	Documento de Diseño, sección 3; ci.yml en 9b4e66b
I-07	El pipeline se integró mediante un PR dirigido a feature/nova-campus (no a main) y sin revisión humana, como excepción deliberada para no dejar el CI sin publicar mientras la fusión a main no estaba disponible	PR #4 (base feature/nova-campus, 0 revisiones); Documento de Diseño, sección 2

Consecuencia documental: el plan de referencia de este informe es el calendario oficial de la ESIT y los requisitos del documento del proyecto. Las inconsistencias se registran para trazabilidad; no modifican el alcance ejecutado.

8. Estado real vs planificado

Base del plan: calendario oficial ESIT (Fase 2: repositorio, integración continua y calidad, del 19 de octubre al 8 de noviembre) y los requisitos del documento del proyecto

Criterio de valoración de los estados:
• Cumplido — existe el entregable completo y su evidencia es verificable mediante enlace, hash o ejecución.
• Parcial — existe parte del entregable o de la evidencia, pero falta un elemento del alcance declarado; la columna «Evidencia» indica cuál.
• No ejecutado — no existe entregable dentro del periodo. Cuando la causa es una dependencia externa, se indica en la misma columna.
• No aplica — la actividad no corresponde a la fase o fue descartada con justificación documentada.

Un estado «Cumplido» exige evidencia enlazada: sin enlace verificable, la actividad se registra como parcial.

Actividad planificada	Responsable	Ejecutada	Evidencia	Estado
Estructura del repositorio y estrategia de	DevOps	Sí	main protegida; feature/*, ci/*; PR #1 y #4	Cumplido

ramas				
Implementación del pipeline CI	DevOps	Sí	ci.yml en 9b4e66b	Cumplido
Validación automática (lint + pruebas)	DevOps	Sí	Run 42549407938 — 39/39	Cumplido
Verificación de arranque (smoke)	DevOps	Sí	Runs 42549407938 y 42543910619 (rama de QA)	Cumplido
Integración de la aplicación en main	Líder de equipo	No	PR #2 sin revisión; fusión no ejecutada en el periodo	No ejecutado
Matriz de pruebas del pipeline	QA	Parcial — matriz entregada el 10/11	Anexo 1; faltan los casos CI-01, CI-03 y CI-04 corregidos (ver 10.1)	Parcial
Organización de evidencias técnicas	QA / DevOps	Sí	Capturas Fig. 1–9 insertadas y citadas; enlaces de ejecución en la sección 4.5	Cumplido
Bitácora de la fase	Líder de equipo	No		No ejecutado
Contenerización (Docker)	Infraestructura	No	Docker es opcional según el documento del proyecto; el Diseño preveía esclarecer su implementación en Fase 2 y no consta asignación explícita. El Dockerfile existe desde la estructura inicial del proyecto (27/09) y permanece vacío (0 bytes)	No ejecutado (justificado)

9. Conclusiones

Conclusiones:

1. El pipeline de integración continua está implementado, versionado y validado sobre el código real de la aplicación, con evidencia verificable en GitHub Actions.
2. La decisión de sustituir la etapa de build por verificación de arranque está justificada técnicamente y respaldada por la suite de pruebas y el smoke test.
3. Además de la bitácora de fase, el único eslabón técnico no evidenciado es la fusión de la aplicación a main, que depende de una aprobación ajena al equipo de desarrollo y DevOps.
4. Se detectó y documentó un defecto real de la suite (D-01: fallo en clon limpio) que el pipeline mitiga con un workaround explícito.

Próximos pasos (Fase 3, 9–22 de noviembre)

Acción	Detalle
Integrar la aplicación en main	No ejecutada en esta corrección; se traslada a la Fase 3 junto con RT-08 y D-03

Implementar cd.yml	Deploy hook de Render mediante RENDER_DEPLOY_HOOK_URL como secreto del repositorio
Health check posterior al despliegue	Consulta a HEALTHCHECK_URL (equivalente real de RF-05)
Activar RT-08	Configurar Validacion del código como check requerido en main
Corregir D-01	El helper de pruebas debe crear work/ con fs.mkdirSync(..., { recursive: true })
Ajustar /version en despliegue	Definir APP_VERSION y GIT_SHA; usar RENDER_GIT_COMMIT como respaldo
Documentar limitación de SQLite	El plan gratuito de Render tiene disco efímero: la base se reinicia en cada despliegue
Completar el Dockerfile	Está vacío (0 bytes) en main y en la rama de la aplicación; sin contenido no puede construirse un despliegue por contenedor. Requiere, además, que la aplicación esté en main (ver 7.1)
Cerrar o aceptar PR #3 y ramas temporales	Higiene del repositorio antes de la siguiente entrega

10. Anexo

Anexo	Contenido	Ubicacion
1	Matriz de pruebas del pipeline con resultados y evidencias (entrega del 10/11/2026)	Documento adjunto y repositorio; nota de revisión en 10.1
2	Registro de comunicaciones citadas (transcripción C-01 a C-10)	Sección 11
3	Evidencias verificables por integrante	Sección 12

10.1 Nota de revisión del Anexo 1 (matriz de pruebas)

La matriz entregada por QA el 10/11 incorpora el caso CI-11 (clon limpio sin preparación manual), que no figuraba en la entrega del 08/11. Durante la revisión de esa entrega se identificaron tres casos que citan ejecuciones del workflow esqueleto de la Fase 0, anteriores a la implementación del pipeline. Se listan para que el lector los contraste con la sección 4.5; el documento de QA no se modifica.

Caso del anexo	Observaciones	Evidencia válida
CI-01	Cita la ejecución del PR #3 (workflow de Fase 0, ~4 s)	Run 42549407938 (PR #2, siete pasos)
CI-03	Cita la «ejecución #6» del PR #2, también del esqueleto	Runs de la sección 4.5; explicación en la sección 4.4
CI-04	Figura como «pendiente de implementar»; ya está implementado y verificado	Run 42549407938

Los demás casos del anexo (CI-05 a CI-09 y CI-11) son consistentes con las ejecuciones registradas en la sección 4.5. Los casos CI-02 y CI-10 figuran correctamente como pendiente y no aplica, respectivamente.

11. Anexo 2 - Registro de comunicaciones citadas

Transcripción de las comunicaciones del equipo que se citan en el informe. Se identifica al emisor por su rol; no se incluyen números de teléfono ni datos personales. Las citas reproducen el texto original y los cortes se indican con [...]. Las filas están ordenadas por código; la fecha de cada comunicación figura en la tabla.

Codigo	Fecha y hora	Emisor (rol)	Extracto	Se cita en
C-01	06/10 21:54	Líder	"Andrew - Desarrollador App Jr. [...] Tarea técnica: Análisis de requerimientos y Modelo de datos [...] Fran - DevOps Jr. [...] Tarea técnica: Diseño conceptual del flujo de automatización"	sección 2; I-05; sección 12
C-02	17/10 21:54	Desarrollador	"Ya subí la primera versión de la app al repositorio" [...] "Me confirman si está todo bien y que les parece"	sección 2
C-03	18/10 13:06	DevOps	"lo que si, es para la fase 2, pero con lo que subiste ya me dio para agregarlo en los docs de analisis y disenho"	sección 2, I-05
C-04	22/10 20:46	DevOps	"El dia de manhana quiero hacer el pr con el contenido del CI, pero primero necesito que le hagan merge a la app"	sección 7.1
C-05	02/11 22:38	Líder	"ya es que empiecen a subir el codigo que tengan a github [...] Aqui les dejo unas tareas para la fase 3 [...] Validar el funcionamiento de los endpoints (/health y /api/quote) de la Calculadora de Cotizaciones [...] verificar que las etapas de Lint, Build y Deploy"	I-01 a I-04; sección 12
C-06	05/11 19:13	DevOps	"Pueden aceptar el PR del Dev"	sección 7.1
C-07	07/11 14:11	DevOps	"no, sin merge de la app no puedo subir el CI"; QA responde dirigiendo la consulta al líder	sección 7.1
C-08	08/11 16:15	QA	"solo herbert tiene permiso para aceptar los pull request verddad"; DevOps responde afirmativamente	sección 7.1
C-09	09/11 19:39	DevOps	"Ahorita mismo no creo que sea viable hacer el merge a main, ya que se construyo todo sobre el pr de Dev, y aparte el ing no pide que creemos nada, sino que documentemos"	sección 7.1

C-10	07/11 21:39	Infraestructura	"ya dejé configurado el servidor en Render con el plan gratuito y las variables de entorno"	sección 12

12. Anexo 3 - Evidencias verificables por integrante

Registro de los artefactos atribuibles a cada integrante según el repositorio, las ejecuciones de GitHub Actions y las asignaciones documentadas (C-01, C-05, C-10). No sustituye los aportes individuales: eso le corresponde a la bitácora de la fase, que cada integrante completa. Conforme al punto 3 de los lineamientos del tutor, cada fila identifica actividades asignadas, entregables y enlaces verificables; cuando no existe evidencia atribuible en el repositorio, se indica expresamente.

Integrante	Actividades asignadas	Artefactos atribuibles	Enlaces verificables
Juan F. Azucena Mejía (Ing. DevOps Jr.)	Diseño conceptual del flujo de automatización; redacción de la sección Modelo de pipeline del Documento de Análisis (C-01)	Documento de Análisis completo (5 secciones); Documento de Diseño (5 secciones + diagrama); implementación y validación del pipeline; registro de defectos D-01 a D-04	Documento de Análisis y Documento de Diseño (repositorio, carpeta docs/fase1/)commits 8d97e62, e1b9bf5, 9b4e66b; runs 42548599241, 42549407938
Andrew Chicas (Desarrollo de aplicación)	Análisis de requerimientos; documentación del endpoint /health (C-01)	Aplicación Nova Campus y suite de pruebas, entregadas el 17/10 como base de la fase; sin commits propios entre el 19/10 y el 08/11	Commit 580c910 (17/10); PR #2
Héctor B. Chipagua Calles (QA / documentación)	Consolidación del Documento de Análisis; definición de criterios de éxito (C-01)	Matriz de pruebas del pipeline (entrega del 10/11); ejecución de casos negativos provocados (lint y pruebas)	Runs 42539880570, 42542545830, 42543910619; matriz en anexo
Diego Alfaro (Infraestructura)	Selección de plataforma de alojamiento; definición de secretos y puertos (C-01)	Configuración del servicio en Render (plan gratuito y variables de entorno), reportada el 07/11; sin artefacto en el repositorio	C-10
Herberth Barrera (Líder de equipo)	Coordinación, asignación de tareas y aprobación de fusiones	Revisión y aprobación del PR #1; asignación de tareas al equipo (C-01, C-05)	PR #1; commit de fusión 2eb85ab

ESCUELA SUPERIOR
de Innovación
y Tecnología

Estancia
Profesional
ESCUELA SUPERIOR DE INNOVACIÓN Y TECNOLOGÍA